

Engineering High-Fidelity Reinforcement Learning Arenas for Algorithmic Trading: A Hybrid Rust-Python Architecture

1. Executive Summary

The convergence of high-frequency trading (HFT) infrastructure and deep reinforcement learning (DRL) represents a critical frontier in modern quantitative finance. While traditional algorithmic trading relies on stochastic calculus and heuristic strategies, the DRL paradigm offers the theoretical capacity to learn optimal policies directly from raw market microstructure. However, a significant "production gap" exists: agents trained in simplified, slow Python environments frequently fail to generalize to the stochastic, latency-sensitive, and adversarial reality of live markets. This report articulates a comprehensive architectural blueprint for bridging this gap by constructing a high-performance "Arena"—a unified simulation and training environment leveraging **Rust** for market physics and **Python** for cognitive policy optimization.

The analysis synthesizes methodologies from high-frequency backtesting, decentralized prediction markets (specifically Polymarket), and sports betting integration to define a universal trading substrate. By utilizing the Foreign Function Interface (FFI) capabilities of **PyO3** and **Maturin**, we can achieve "zero-copy" memory sharing, allowing Python-based neural networks (PyTorch, TensorFlow) to interact with Rust-based order book simulations at speeds exceeding 10,000 steps per second. This hybrid approach resolves the bottleneck of data serialization that plagues pure Python implementations.

Furthermore, this report integrates advanced observability patterns. It moves beyond simple profit-and-loss (PnL) curves to encompass real-time 3D visualization of limit order books via **ReRun.io** and the topological analysis of neural network loss landscapes. Understanding the geometry of the loss surface—specifically the sharpness or flatness of local minima—provides critical insights into the robustness of trading agents. Through this multi-layered analysis, we establish a rigorous engineering standard for developing autonomous agents capable of navigating the complex dynamics of crypto CLOBs (Central Limit Order Books), binary prediction markets, and odds-based gambling environments.

2. The Computational Substrate: Rust-Python Hybridization

The fundamental challenge in applying reinforcement learning to financial markets is sample

inefficiency. Modern off-policy algorithms like Soft Actor-Critic (SAC) or on-policy methods like Proximal Policy Optimization (PPO) often require billions of interaction steps to converge on a profitable policy, particularly when the observation space includes high-dimensional inputs such as the full depth of a limit order book. Traditional Python-based environments, such as those built strictly on pandas or pure Python gym classes, suffer from significant latency due to the Global Interpreter Lock (GIL) and the overhead of dynamic typing. To replicate the physics of a market—processing millions of quote updates, matching orders, and calculating margin requirements—a systems-level language is required.

2.1 The Architectural Imperative for Rust

Rust has emerged as the premier candidate for the "simulation core" of trading arenas due to its unique combination of memory safety without garbage collection and C++-level performance. In an RL context, the simulation engine acts as the "Server" and the agent as the "Client." The server must maintain the canonical state of the market, reconstructing order books from incremental tick data and strictly enforcing time-priority and price-priority matching rules.

The ownership model of Rust is particularly advantageous for handling financial state. In a multithreaded backtesting environment, where multiple "universes" or parallel environments are stepped simultaneously to fill a training batch, Rust's compile-time guarantees prevent data races. This allows for the safe implementation of **vectorized environments**—a technique where a single Python process steps N Rust environments in parallel via a thread pool, maximizing the throughput of the GPU consuming the observations. Frameworks like **NautilusTrader** have demonstrated the efficacy of this approach, utilizing Rust for the event-driven core while exposing Python bindings for strategy definition.¹ Similarly, **HftBacktest** utilizes Rust to handle the heavy lifting of recreating local order queues and simulating latency, ensuring that the backtest accounts for the physical delay between signal generation and order execution.³

2.2 Bridging the Divide: PyO3 and Maturin

To make a Rust-based simulation accessible to the rich ecosystem of Python RL libraries (such as Stable Baselines3, CleanRL, or Ray RLlib), we must construct a high-performance bridge. **PyO3** is the de facto standard crate for writing native Python extensions in Rust. It handles the complexity of the Python C-API, allowing Rust structs to be exposed as Python classes and Rust functions to be callable as Python methods.⁵

Complementing PyO3 is **Maturin**, a build system that simplifies the compilation and packaging of Rust crates into Python wheels. Maturin automates the linking process and creates distinct binary distributions for different platforms, effectively treating the Rust simulation as a standard pip-installable library.⁵ This workflow enables a development cycle where the performance-critical "Arena" is compiled once, and the Python-based "Agent" can

be iterated upon rapidly without recompilation.

2.2.1 The Zero-Copy Optimization Pattern

A critical architectural constraint in high-frequency RL is the cost of data transfer. In a naive implementation, when the Python agent requests an observation (e.g., the current state of the order book), the Rust engine creates a copy of the data, serializes it into a Python list or NumPy array, and passes it across the boundary. For a tensor of shape $(100, 10, 2)$ representing market depth, performing this copy operation millions of times results in substantial overhead.

The solution is the implementation of **zero-copy views** using the `numpy` crate within the Rust ecosystem. This technique allows the Rust simulation to allocate a contiguous block of memory for the observation buffer (e.g., a `Vec<f32>` or an `ndarray::Array`). Instead of copying this data, the Rust code creates a `PyArray` reference that points directly to this memory address. The Python interpreter, specifically the NumPy library, treats this memory as its own, allowing the DRL framework (PyTorch or TensorFlow) to read directly from the simulation's state without duplication.⁸

This pattern effectively eliminates the serialization bottleneck. The Rust engine updates the market state in place, and the Python agent reads the updated state immediately. This architecture mirrors the memory management strategies used in high-performance computer vision and robotics simulations, bringing those efficiency gains to the financial domain.¹⁰

2.3 Managing the Global Interpreter Lock (GIL)

When training RL agents, particularly with on-policy algorithms like PPO, data collection (stepping the environment) often alternates with gradient updates. To maximize throughput, the simulation should ideally run on background threads while the GPU performs the backward pass. However, Python's GIL notoriously prevents true parallelism for CPU-bound tasks.

Rust interacts with the GIL through PyO3's `Python::allow_threads` mechanism. This allows the Rust code to release the GIL before entering a computationally intensive section (such as matching orders in a complex CLOB or parsing a large parquet file of historical data) and re-acquire it only when returning results to Python. This capability is crucial for the "Arena" concept; it permits the heavy simulation logic to execute on all available CPU cores via Rust's **Rayon** or **Tokio** schedulers, completely independent of the Python interpreter's lock.⁶ This parallelism is instrumental when simulating hundreds of uncorrelated assets simultaneously, such as a portfolio of prediction market contracts or a universe of crypto pairs.

3. Market Microstructure Simulation: The Arena

Physics

The validity of an RL agent's training depends entirely on the fidelity of the simulation. If the environment simplifies execution logic—assuming infinite liquidity or zero latency—the agent will learn to exploit these simulator artifacts rather than genuine market inefficiencies. Therefore, the Arena must implement asset-specific engines that replicate the distinct "physics" of different market types.

3.1 High-Frequency Trading and Crypto CLOBs

For cryptocurrency and traditional equity markets, the dominant microstructure is the Central Limit Order Book (CLOB). To train agents for these markets, the simulation must go beyond "bar-based" (OHLC) backtesting and implement **tick-by-tick** resolution.

HftBacktest serves as a primary reference implementation for this domain. It constructs a full local order book from incremental feed updates, differentiating between "Maker" (limit) and "Taker" (market) orders.³ A critical feature for RL training is the **Queue Position Model**. When an agent places a limit order at the best bid, it does not fill immediately; it joins the back of the queue. The simulator must track the volume of trades hitting the bid to determine exactly when the agent's order is reached and executed. HftBacktest supports advanced queue models, including probability-based estimates for Level 2 data (Market-by-Price) and precise FIFO tracking for Level 3 data (Market-by-Order).¹¹

NautilusTrader extends this by providing an event-driven architecture that handles complex order types (Iceberg, Good-Till-Cancel, Immediate-or-Cancel) and diverse execution venues. Its Rust-based core ensures that the logic used during the backtest phase is identical to the logic used in live deployment, eliminating the "implementation risk" where a strategy behaves differently in production due to code discrepancies.¹ For the Arena, wrapping NautilusTrader's execution engine in a Gymnasium interface allows the RL agent to learn sophisticated execution strategies, such as splitting large orders to minimize market impact or probing for hidden liquidity.

3.2 Decentralized Prediction Markets: The Polymarket Engine

Prediction markets like **Polymarket** introduce unique microstructural elements that differ from standard financial instruments. Polymarket operates on the Polygon blockchain (and formerly Gnosis), using the **Conditional Token Framework (CTF)**. Understanding and simulating this framework is essential for training agents to trade binary outcomes.

The Polymarket Arena must simulate a **hybrid-decentralized** architecture. While the order matching happens off-chain via a CLOB (managed by an operator to ensure speed), settlement is on-chain.¹³ The simulation must replicate the **Negative Risk** (NegRisk) properties of these markets. In a standard exchange, buying two mutually exclusive assets (e.g., "Trump Yes" and "Biden Yes") would require full collateral for both. In Polymarket's

NegRisk system, the exchange recognizes that these outcomes cannot both occur, allowing for cross-collateralization and reduced capital requirements. An RL agent unaware of NegRisk will drastically underestimate its buying power and fail to optimize capital efficiency.¹³

Furthermore, the simulation must handle the **Merge** and **Split** mechanics of CTF. Collateral (USDC) can be split into a complete set of outcome tokens (e.g., 1 "Yes" + 1 "No"), and conversely, a complete set can be merged back into collateral. This creates an arbitrage bound: the sum of the prices of all mutually exclusive outcomes should theoretically equal \$1.00. An RL agent can be trained specifically to exploit deviations from this parity, effectively learning to perform risk-free arbitrage. The Rust simulation must validate these complex atomic actions, ensuring the agent interacts with the market physics correctly.¹³

3.3 Sports Betting and Episodic Odds

Sports betting markets fundamentally differ from continuous double auctions. They are primarily **Quote-Driven** or odds-based, where a bookmaker sets the price (odds) and the agent decides whether to accept. While exchange-based betting (like Betfair) exists, the vast majority of liquidity is in fixed-odds bookmakers.

The "Sports Arena" simulation relies on historical odds snapshots, which can be sourced from providers like **The Odds API** or **SportsDataIO**.¹⁶ Unlike the millisecond-level updates of HFT, sports betting is episodic with long horizons. The state transitions occur when odds change (due to injury news or betting volume) or when the game concludes.

A crucial concept for the RL agent to learn in this domain is **Closing Line Value (CLV)**. Empirical research suggests that the most reliable predictor of long-term profitability is not just winning bets, but consistently betting at odds better than the "closing line" (the odds at the moment the game starts). The simulation environment should essentially track CLV as an auxiliary reward signal, encouraging the agent to identify value early in the market cycle before information becomes efficient.¹⁷ The simulation must also handle the discrete nature of sports events—markets close, resolve, and pay out, unlike the perpetual trading of crypto assets.

4. The Reinforcement Learning Architecture

With the high-performance Rust simulation established, the focus shifts to the design of the "Brain"—the RL agent. This involves rigorously defining the observation space, action space, and reward functions to map financial objectives into learnable tasks.

4.1 Tensorizing the Market: Observation Space Engineering

The observation space is the agent's perceptual interface. Passing raw prices is rarely

effective due to the non-stationary nature of financial time series. The Rust engine must preprocess market data into normalized tensors before passing them to the Python agent.

For order book markets, the standard representation is a **LOB Tensor** of shape $(K, 2, 2)$, representing the top K price levels for Bids and Asks, with channels for Price and Size.

- **Normalization:** Prices should be encoded relative to the mid-price (e.g., percent deviation) to make the input scale-invariant. Sizes should be log-normalized to handle the heavy-tailed distribution of order quantities.¹⁸
- **Market Imbalance:** Rust-calculated metrics such as **Order Book Imbalance (OBI)** or **Volume-Adjusted Mid Price (VAMP)** provide high-level signal features that are expensive for a neural network to infer from raw data alone. Including these as explicit features accelerates convergence.¹⁹
- **Regime Awareness:** Financial markets exhibit distinct regimes (e.g., mean-reverting vs. trending, low vs. high volatility). The Arena can integrate a **Regime Signal**, calculated via Hidden Markov Models (HMM) or clustering algorithms on historical data, into the observation space. This "context vector" allows the agent to modulate its policy—for example, reducing position sizes during high-volatility regimes.²⁰

4.2 The Decision Matrix: Action Space Formulation

The formulation of the action space dictates the complexity of the learning problem and the choice of algorithm.

4.2.1 Discrete vs. Continuous Actions

- **Discrete Action Space:** The agent selects from a finite set of moves: ````. This setup is compatible with value-based algorithms like **DQN** (Deep Q-Networks). It works well for directional betting where the stake size is fixed (e.g., "Bet 1 unit"). However, it lacks the nuance required for precise portfolio management or execution algorithms.¹⁸
- **Continuous Action Space:** The agent outputs continuous values, typically a vector representing the target portfolio allocation (e.g., $[0.5, 0.2, 0.3]$ for a 3-asset portfolio). This requires policy-gradient methods like **PPO** (Proximal Policy Optimization) or **SAC** (Soft Actor-Critic). Continuous spaces allow the agent to learn **Position Sizing**, which is critical for risk management. In the context of the Kelly Criterion, a continuous agent can learn to adjust its bet size proportional to its perceived edge, theoretically optimizing the geometric growth rate of the portfolio.²²

4.2.2 Parameterized Action Spaces

For HFT and complex markets like Polymarket, a **hybrid** or **parameterized** action space is often superior. The agent first selects a discrete action type (e.g., Limit Buy, Market Sell) and then outputs continuous parameters for that action (e.g., Price Offset, Quantity). Handling this in Gym requires using `gymnasium.spaces.Tuple` or `Dict` spaces, and often necessitates specialized architectures like the **Q-PAMDP** algorithm to handle the hierarchical decision

process.²⁴

4.3 Reward Function Engineering

The reward function is the most potent lever in RL design. A naive reward function based solely on "Net Profit" often leads to reckless gambling strategies that eventually blow up the account.

- **Risk-Adjusted Rewards:** To encourage sustainable trading, the reward should reflect risk-adjusted returns. Metrics like the **Sharpe Ratio** or **Sortino Ratio** (which penalizes only downside volatility) can be calculated over a rolling window and used as the reward signal. This penalizes the agent for taking excessive risks to achieve small gains.²⁰
- **Transaction Cost Awareness:** The environment must strictly deduct transaction fees (slippage + taker fees) from the reward. Without this, agents in high-frequency environments will inevitably learn to "churn"—trading rapidly to capture microscopic noise that disappears once fees are applied.¹⁸
- **Drawdown Penalties:** Explicitly penalizing maximum drawdown (the peak-to-trough decline) conditions the agent to prioritize capital preservation. This can be implemented as a negative reward proportional to the depth of the current drawdown.¹⁸

5. Visualization and Observability: The Glass Box

Deep Reinforcement Learning models are often criticized as "black boxes." To engineer reliable trading agents, we must implement deep observability pipelines that allow us to inspect the agent's internal state and the environment's dynamics in real-time.

5.1 Real-Time Multimodal Telemetry with Rerun

Rerun.io is a cutting-edge visualization SDK that revolutionizes how we monitor RL training. Unlike simple scalar loggers (like TensorBoard), Rerun handles multimodal data—tensors, 3D geometry, images, and text—synchronized on a unified timeline.²⁵

In the context of the Trading Arena, Rerun enables the construction of a **3D Market Replay**.

- **The Terrain:** The Limit Order Book can be visualized as a 3D terrain where the X-axis is Price, the Y-axis is Time, and the Z-axis (height) is Liquidity/Depth. This visualization intuitively highlights "liquidity walls" and order flow dynamics that are invisible in 2D charts.²⁷
- **Agent Trajectory:** The agent's actions can be overlaid on this terrain as 3D objects (e.g., spheres or arrows), color-coded by side (Buy/Sell) and sized by confidence or volume. This allows researchers to visually debug the agent's behavior—for example, verifying if the agent correctly identifies and reacts to a liquidity vacuum.²⁵
- **Internal State:** Rerun can also visualize the internal activations of the neural network (e.g., attention maps in a Transformer-based agent) alongside the market data, helping to

correlate specific market patterns with the agent's focus.²⁸

5.2 Loss Landscape Analysis

Beyond visualizing the market, we must visualize the *learning process* itself. The geometry of the neural network's loss landscape determines the stability and generalization capability of the trained agent.

- **The Theory:** Research by Li et al. (2018) suggests that "flat" minima in the loss landscape generalize better than "sharp" minima. In trading, a sharp minimum implies that a small change in market distribution could cause the agent's performance to collapse.²⁹
- **Implementation:** We can visualize this geometry by capturing the model's weights at various checkpoints. Using **Principal Component Analysis (PCA)**, we project the high-dimensional trajectory of the weights into a 2D or 3D space. Plotting the loss value over this 2D plane reveals the "terrain" the optimizer is traversing.
- **Diagnostics:** A chaotic, high-variance trajectory suggests the learning rate is too high or the batch size too small. A trajectory that circles a valley without entering indicates poor convergence. Libraries like loss-landscapes (PyTorch) can be integrated into the python training loop to generate these visualizations periodically, providing a "health check" on the agent's optimization journey.³⁰

6. Data Infrastructure and Standardization

The fuel for this entire apparatus is data. The Arena requires a robust pipeline to ingest, normalize, and serve historical data from diverse sources.

6.1 Heterogeneous Data Ingestion

- **Crypto/HFT:** Data is typically sourced in **Parquet** or **HDF5** format from providers like **Databento** or **Binance Vision**. This data is massive (terabytes of tick data). The Rust engine utilizes Apache Arrow (via the polars or arrow crates) to map these files into memory efficiently, allowing for fast random access during training.¹
- **Sports/Gambling:** Data often comes as JSON or CSV snapshots from APIs. The pipeline must normalize these into a time-series format, aligning odds from multiple bookmakers to simulate the "Best Execution" environment where the agent can shop for the best lines.¹⁶
- **Prediction Markets:** Data from Polymarket involves parsing blockchain event logs (via The Graph or Covalent) to reconstruct the history of Conditional Token balances and automated market maker states. This data must be cleaned to account for blockchain reorgs and missing blocks.¹⁵

6.2 The Universal Interface

To allow the RL agent to transfer knowledge between domains (e.g., pre-training on high-frequency crypto data and fine-tuning on sports betting), the Arena should implement a **Standardized Interface**. This involves normalizing all inputs to a standard distribution (Z-score normalization) and mapping all action spaces to a common range (e.g., $[-1, 1]$). This supports **Transfer Learning** and **Meta-Learning**, where a single "Foundation Agent" learns the universal dynamics of risk and reward before specializing in a specific asset class.¹⁸

7. Implementation Roadmap

Phase 1: Core Substrate Development

- Initialize the Rust workspace and define the OrderBook and MatchingEngine structs.
- Implement the HFT logic using HftBacktest patterns for queue simulation.
- Develop the PolymarketEngine with CTF Merge/Split logic and NegRisk accounting.

Phase 2: Python Binding and Gym Integration

- Use PyO3 to wrap the Rust engines.
- Implement the Gymnasium traits (reset, step, render) in Python, calling the Rust backend.
- Verify zero-copy memory access using numpy views.

Phase 3: Observability Suite

- Integrate rerun-sdk into the Python training loop.
- Implement the 3D Order Book visualizer.
- Set up the loss-landscapes workflow for weight trajectory analysis.

Phase 4: Training and Evaluation

- Train baseline agents (PPO, SAC) on historical data.
- Use Criterion.rs to benchmark simulation throughput, targeting >10,000 SPS.
- Evaluate agents using "Out-of-Sample" testing and rigorous Sharpe Ratio analysis.

8. Conclusion

The construction of a Reinforcement Learning Arena for gambling and investing is a multidisciplinary engineering feat that demands the highest levels of performance and rigor. By fusing the systems-level efficiency of **Rust** with the cognitive flexibility of **Python**, we can create a simulation environment that breaks the "Sim-to-Real" barrier.

This hybrid architecture, powered by **PyO3** and **Zero-Copy** memory sharing, provides the throughput necessary for modern DRL algorithms to converge. The integration of domain-specific physics engines—from **HftBacktest**'s queue models to **Polymarket**'s

conditional token logic—ensures that the agents learn robust, realistic strategies. Finally, by wrapping this system in advanced observability layers like **Rerun.io** and **Loss Landscape** analysis, we transform the training process from a black box into a transparent scientific laboratory. This report provides the roadmap for building not just a backtester, but a foundational platform for the future of autonomous financial intelligence.

Table 1: Technology Stack Summary

Layer	Component	Technology/Library	Function
Agent	Brain	Python (PyTorch, Stable Baselines3)	Neural Network Policy, PPO/SAC Algorithms
Interface	Bridge	PyO3, Maturin, Gymnasium	FFI, Python Bindings, RL Standard API
Simulation	Core	Rust (NautilusTrader, HftBacktest)	Market Physics, Order Matching, State Management
Data	Memory	NumPy, Rust ndarray, Apache Arrow	Zero-Copy Shared Memory, Parquet Loading
Vis	Telemetry	Rerun.io, Matplotlib	Real-time 3D Replay, Tensor Visualization
Diagnostics	Analysis	Criterion.rs, loss-landscapes	Benchmarking, Weight Trajectory Analysis

This architecture represents the current state-of-the-art in open-source quantitative infrastructure, democratizing access to the tools required to compete in the world's most competitive markets.

Works cited

1. nautechsystems/nautilus_trader: A high-performance algorithmic trading platform and event-driven backtester - GitHub, accessed January 1, 2026, https://github.com/nautechsystems/nautilus_trader
2. NautilusTrader, accessed January 1, 2026, <https://nautilustrader.io/>
3. High-Frequency Trading Backtesting Tool - HftBacktest - Read the Docs, accessed January 1, 2026, <https://hftbacktest.readthedocs.io/en/py-v2.1.0/index.html>
4. hftbacktest - crates.io: Rust Package Registry, accessed January 1, 2026, <https://crates.io/crates/hftbacktest>
5. Tutorial - Maturin User Guide, accessed January 1, 2026, <https://www.maturin.rs/tutorial.html>
6. Introduction - PyO3 user guide, accessed January 1, 2026, <https://pyo3.rs/>
7. Writing python extensions never been easier... with Rust and PyO3 - Reddit, accessed January 1, 2026, https://www.reddit.com/r/Python/comments/125q9vo/writing_python_extensions_never_been_easier_with/
8. PyArray in numpy::array - Rust - Docs.rs, accessed January 1, 2026, <https://docs.rs/numpy/latest/numpy/array/struct.PyArray.html>
9. Need some help with Pyo3/ numpy/ Rust shared memory. - Reddit, accessed January 1, 2026, https://www.reddit.com/r/rust/comments/it9wcd/need_some_help_with_py3_numpy_rust_shared_memory/
10. How to bind (Python + NumPy) with (Rust + Narray) | by Jonathan Österberg - ITNEXT, accessed January 1, 2026, <https://itnext.io/how-to-bind-python-numpy-with-rust-ndarray-2efa5717ed21>
11. High-Frequency Trading Backtesting Tool - HftBacktest - Read the Docs, accessed January 1, 2026, <https://hftbacktest.readthedocs.io/en/v1.8.4/>
12. HftBacktest — hftbacktest, accessed January 1, 2026, <https://hftbacktest.readthedocs.io/>
13. CLOB Introduction - Polymarket Documentation, accessed January 1, 2026, <https://docs.polymarket.com/developers/CLOB/introduction>
14. Historical Timeseries Data - Polymarket Documentation, accessed January 1, 2026, <https://docs.polymarket.com/developers/CLOB/timeseries>
15. Polymarket Data API - Bitquery Documentation, accessed January 1, 2026, <https://docs.bitquery.io/docs/examples/polymarket-api/>
16. Historical Odds Data - SportsDataIO, accessed January 1, 2026, <https://sportsdata.io/historical-odds>
17. Historical Sports Odds Data API, accessed January 1, 2026, <https://the-odds-api.com/historical-odds-data/>
18. Portfolio Optimization with Reinforcement Learning (PPO + LSTM) | by Tinotenda M, accessed January 1, 2026, <https://medium.com/@tinotendamarufetu/portfolio-optimization-with-reinforcement-learning-ppo-lstm-be18b7340731>
19. nkaz001/hftbacktest: Free, open source, a high frequency ... - GitHub, accessed January 1, 2026, <https://github.com/nkaz001/hftbacktest>

20. Adaptive and Regime-Aware RL for Portfolio Optimization - arXiv, accessed January 1, 2026, <https://arxiv.org/html/2509.14385v1>
21. Discretizing Continuous Action Space with Unimodal Probability Distributions for On-Policy Reinforcement Learning - arXiv, accessed January 1, 2026, <https://arxiv.org/html/2408.00309v1>
22. Reinforcement Learning in Continuous Action Spaces | by Amit Yadav - Medium, accessed January 1, 2026, <https://medium.com/@amit25173/reinforcement-learning-in-continuous-action-spaces-4fc60897fa55>
23. The Best Tools for Reinforcement Learning in Python You Actually Want to Try - Neptune.ai, accessed January 1, 2026, <https://neptune.ai/blog/the-best-tools-for-reinforcement-learning-in-python>
24. How to handle variable action spaces in OpenAI Gym environments, accessed January 1, 2026, <https://community.latenode.com/t/how-to-handle-variable-action-spaces-in-openai-gym-environments/23372>
25. Send from Rust - Rerun, accessed January 1, 2026, <https://rerun.io/docs/getting-started/data-in/rust>
26. Rerun — Rerun, accessed January 1, 2026, <https://www.rerun.io/>
27. Raw mesh - Rerun, accessed January 1, 2026, https://rerun.io/examples/feature-showcase/raw_mesh
28. Rerun.io: My Deep Dive into the Go-To Visualizer for Physical AI, accessed January 1, 2026, <https://skywork.ai/skypage/en/Rerun.io-My-Deep-Dive-into-the-Go-To-Visualizer-for-Physical-AI/1975249775198138368>
29. Visualizing the Loss Landscape of Neural Nets - NIPS papers, accessed January 1, 2026, <https://papers.neurips.cc/paper/7875-visualizing-the-loss-landscape-of-neural-nets.pdf>
30. Approximating neural network loss landscapes in low-dimensional parameter subspaces for PyTorch - GitHub, accessed January 1, 2026, <https://github.com/marcellodebernardi/loss-landscapes>
31. PCA of high dimensional random walks with comparison to neural network training - NeurIPS, accessed January 1, 2026, <http://papers.neurips.cc/paper/8232-pca-of-high-dimensional-random-walks-with-comparison-to-neural-network-training.pdf>
32. Integrating Custom Data — hftbacktest - Read the Docs, accessed January 1, 2026, <https://hftbacktest.readthedocs.io/en/v1.8.4/tutorials/Integrating%20Custom%20Data.html>
33. jordantete/OddsHarvester: A python app designed to scrape and process sports betting data directly from oddsportal.com - GitHub, accessed January 1, 2026, <https://github.com/jordantete/OddsHarvester>
34. Polymarket Orderbook | Graph Explorer, accessed January 1, 2026, <https://thegraph.com/explorer/subgraphs/7fu2DWYK93ePfzB24c2wrP94S3x4LGH>

[UrQxphhoEypyY?view=Query&chain=arbitrum-one](#)

35. The Ultimate Guide to Reinforcement Learning Frameworks and Libraries 2024 - Medium, accessed January 1, 2026,
<https://medium.com/@digitalconsumer777/the-ultimate-guide-to-reinforcement-learning-frameworks-and-libraries-2024-b1da545f282f>